

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

"Сибирский государственный университет телекоммуникаций и
информатики"

Кафедра телекоммуникационных систем и вычислительных средств
(ТС и ВС)

РАСЧЁТНО-ГРАФИЧЕСКАЯ РАБОТА

по дисциплине
"Web-технологии"

по теме:

СОЗДАНИЕ ТЕСТОВОЙ ВИДЕОПЛАТФОРМЫ С
ИСПОЛЬЗОВАНИЕМ ТЕХНОЛОГИЙ PYTHON ДЛЯ
БЭКЕНДА, REACT JS ДЛЯ ФРОНТЕНДА И FIGMA ДЛЯ
ДИЗАЙНА

Студент:

Группа ИКС-531

Д.А. Язиков

Преподаватель:

ст. преподаватель

А.В. Андреев

Новосибирск 2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ.....	5
1.1 Архитектура клиент-серверного веб-приложения	5
1.2 Серверные технологии: Python, Django и REST Framework	5
1.3 Реальное время: ASGI, Django Channels и WebSocket.....	6
1.4 Принцип синхронизации воспроизведения.....	6
1.5 Клиентские технологии: React и React Router	7
1.6 Макет интерфейса в Figma	7
2 АРХИТЕКТУРА И СРЕДА РАЗРАБОТКИ	9
2.1 Общая архитектура системы	9
2.2 Модель данных	10
2.3 REST-API	10
2.4 WebSocket-протокол комнаты	11
2.5 Окружение разработки и развёртывания.....	11
3 ПРАКТИЧЕСКАЯ РЕАЛИЗАЦИЯ.....	12
3.1 Регистрация и вход	12
3.2 Загрузка и потоковая отдача видео	13
3.3 Лента, просмотр и профиль	14
3.4 Комнаты и синхронизация воспроизведения	15
3.5 Живой чат и вкладка <<Вопрос/ответ>>.....	17
3.6 Клиентское приложение и дизайн-система	17
3.7 Тестирование	18
3.8 Развёртывание на сервере	19
3.9 Трудности и пути их решения.....	20
ЗАКЛЮЧЕНИЕ	21

ВВЕДЕНИЕ

Видеоплатформы — один из самых нагруженных классов веб-приложений: они объединяют загрузку и хранение больших файлов, потоковую отдачу видео, управление пользователями и обширную интерактивную часть. На примере такого приложения удобно показать полный современный стек веб-разработки: серверный API на **Python**, клиентское одностраничное приложение на **React**, заранее спроектированный в **Figma** интерфейс и развёртывание готового продукта на реальном сервере.

Расчётно-графическая работа посвящена созданию тестовой видеоплатформы **blxck.hub**. Помимо базовых возможностей видеохостинга (регистрация, загрузка роликов, лента, страница просмотра) в проект заложена сигнатурная функция — **синхронные комнаты совместного просмотра**. Пользователь создаёт комнату под любое видео и делится ссылкой; все участники смотрят ролик синхронно — плеер ведущего является источником истины, а плееры зрителей автоматически подстраиваются под него. Рядом с видео работает живой чат с лайками на сообщениях и вкладка <<Вопрос / ответ>>. Эта функция опирается на двунаправленный обмен по протоколу **WebSocket** и делает проект содержательнее обычного учебного CRUD-приложения.

Выбор именно совместного просмотра отражён и в макете Figma: его центральный экран — видеоплеер с боковой панелью чата и вкладкой вопросов. Дизайн-система макета (поля ввода с состояниями, карточки сообщений, кнопки) перенесена в код, интерфейс выдержан в тёмной теме бренда **blxck.hub**.

Цель работы — спроектировать и реализовать полнофункциональную видеоплатформу на современном стеке веб-технологий, обеспечить взаимодействие серверной и клиентской частей через REST- и WebSocket-API и развернуть готовый продукт на сервере с доступом по защищённому протоколу HTTPS.

Задачи:

1. Спроектировать в Figma макет интерфейса: цветовую палитру, типографику, состав экранов и компонентов.

2. Разработать серверную часть на Django: модель данных, авторизацию по паролю и гостевые аккаунты с выдачей JWT-токенов, загрузку и потоковую отдачу видео, REST-API.
3. Реализовать комнаты совместного просмотра и живой чат на Django Channels поверх WebSocket.
4. Разработать клиентское приложение на React с маршрутизацией, интеграцией с API и реализацией компонентов дизайн-системы.
5. Покрыть тестами серверную и клиентскую части.
6. Развернуть платформу на сервере за обратным прокси Caddy, оформить документацию API в Git.

Инструменты и средства:

- Python 3 с фреймворками Django, Django REST Framework и Django Channels — серверная часть и REST/WebSocket-API;
- Daphne — ASGI-сервер для обслуживания HTTP и WebSocket;
- Redis — слой обмена сообщениями Channels;
- PostgreSQL — база данных продакшена, SQLite — база данных локальной разработки;
- Node.js, React, React Router, Vite — клиентское одностраничное приложение;
- Figma — источник макета и дизайн-системы интерфейса;
- Docker, docker-compose, обратный прокси Caddy с автоматическим TLS Let's Encrypt — развёртывание на собственном сервере;
- pytest и Vitest — тестирование серверной и клиентской частей;
- Git — контроль версий и документация проекта.

Готовая платформа публикуется по адресу <https://blxckhub.server34.netcraze.club>.

1 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

1.1 Архитектура клиент-серверного веб-приложения

Современное веб-приложение строится по схеме <<клиент — сервер>> с чётким разделением ответственности. **Сервер** (бэкенд) хранит данные, реализует бизнес-логику и предоставляет программный интерфейс — **API**. **Клиент** (фронтенд) выполняется в браузере пользователя, отвечает за интерфейс и обращается к серверу за данными.

В проекте применяется модель **одностраничного приложения** (*Single Page Application*, SPA): браузер один раз загружает клиентский код, а далее не перезагружает страницу целиком — при переходах он лишь запрашивает данные у API и перерисовывает нужные части интерфейса. Такой подход даёт отзывчивый интерфейс и снимает с сервера задачу генерации HTML.

Взаимодействие клиента и сервера в blxck.hub идёт по двум каналам:

- **REST-API** поверх HTTP — запрос-ответные операции: регистрация, загрузка видео, получение ленты, создание комнаты;
- **WebSocket** — двунаправленный постоянный канал для событий реального времени: синхронизация плеера и живой чат.

1.2 Серверные технологии: Python, Django и REST Framework

Django — высокоуровневый веб-фреймворк на языке Python. Он предоставляет объектно-реляционное отображение (**ORM**) для работы с базой данных через классы-модели, систему миграций схемы, готовую панель администратора и подсистему аутентификации.

Django REST Framework (DRF) — надстройка над Django для построения REST-API. Ключевые понятия DRF: *сериализаторы*, описывающие преобразование объектов модели в JSON и обратно с валидацией; *представления* и *наборы представлений* (*ViewSet*), реализующие обработку запросов; *маршрутизаторы*, автоматически строящие URL-адреса для типовых CRUD-операций.

Аутентификация построена на **JWT** (*JSON Web Token*) — подписанных сервером токенах, которые клиент хранит у себя и прикладывает к каждому запросу в заголовке `Authorization`. Токен *access* короткоживущий (30 минут) и удостоверяет личность; токен *refresh* живёт дольше (7 дней) и служит для получения нового *access*-токена без повторного входа. Библиотека `djangorestframework-simplejwt` реализует выпуск и проверку токенов.

1.3 Реальное время: ASGI, Django Channels и WebSocket

Классический Django работает по синхронному интерфейсу **WSGI**, который обслуживает только запрос-ответные HTTP-обращения. Для постоянных соединений нужен асинхронный интерфейс **ASGI** и ASGI-сервер — в проекте используется Daphne.

WebSocket — протокол двусторонней связи: после HTTP-рукопожатия с апгрейдом протокола соединение остаётся открытым, и сервер может отправлять данные клиенту в любой момент, не дожидаясь запроса. Это и нужно для синхронного просмотра и чата.

Django Channels расширяет Django поддержкой WebSocket. Логика соединения описывает *консьюмер*; консьюмеры объединяются в *группы* через общий *слой каналов* (*channel layer*). При рассылке в группу сообщение получают все её участники. В продакшене слоем каналов служит Redis, что позволяет нескольким процессам обслуживать одну группу; при локальной разработке достаточно встроенного слоя в памяти процесса.

1.4 Принцип синхронизации воспроизведения

Синхронный просмотр построен на модели *авторитета ведущего*: комната хранит состояние плеера — позицию, флаг воспроизведения и серверную отметку времени последнего изменения. Команды `play`, `pause` и `seek` принимаются только от ведущего. Текущая позиция в любой момент вычисляется как сумма сохранённой позиции и времени, прошедшего с момента её фиксации (если ролик воспроизводится).

Плеер зрителя при подключении получает состояние комнаты и периодически сверяет свою позицию с расчётной. Небольшое расхождение

устраняется плавным изменением скорости воспроизведения, значительное — резкой перемоткой. Так компенсируются задержки сети и буферизация.

1.5 Клиентские технологии: React и React Router

React — библиотека для построения пользовательских интерфейсов из *компонентов* — переиспользуемых строительных блоков со своим состоянием. React хранит виртуальное представление DOM и при изменении состояния обновляет только изменившиеся узлы реального дерева.

React Router реализует клиентскую маршрутизацию: сопоставляет адрес в строке браузера с компонентом-страницей без перезагрузки. Сборку и средой разработки обеспечивает **Vite** — быстрый сборщик с горячей заменой модулей.

1.6 Макет интерфейса в Figma

Figma — браузерный редактор интерфейсов. До начала разработки в Figma спроектирован макет интерфейса платформы: лента видео, экраны регистрации и входа, страница просмотра, комната совместного просмотра с боковой панелью чата и вкладкой <<Вопрос / ответ>>, профиль пользователя (рисунок 1). Параллельно проработана дизайн-система — поля ввода, кнопки, карточки сообщений и видео, — переиспользуемая на всех экранах.

Интерфейс выдержан в тёмной теме с индигово-фиолетовым акцентным цветом бренда `blxsk.hub`. Структура экранов и состав компонентов перенесены из макета в код напрямую. Конкретная схема авторизации — вход по имени пользователя и паролю, гостевые аккаунты — описана в главе 3.

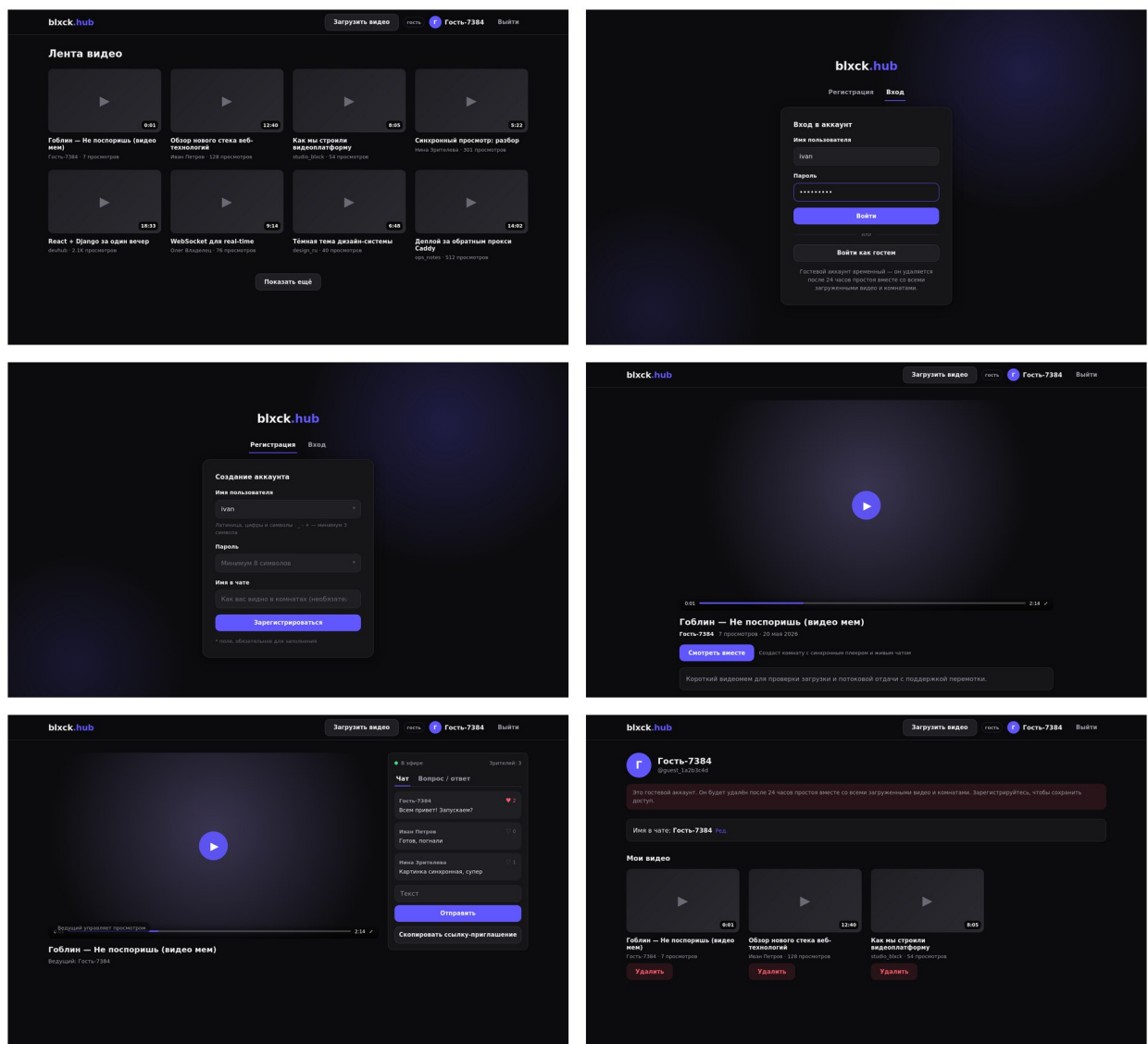


Рисунок 1 — Макет интерфейса blxck.hub в Figma

2 АРХИТЕКТУРА И СРЕДА РАЗРАБОТКИ

2.1 Общая архитектура системы

Платформа `blxck.hub` состоит из четырёх взаимодействующих частей: обратного прокси, клиентского приложения, серверного приложения и хранилищ данных. Запрос пользователя проходит следующий путь:

```
[ Browser ]
  |
  | HTTPS
  v
[ Caddy ]   <-- reverse proxy + TLS
  |
  v
[ nginx ]   <-- SPA, /media, /static
  |         /api/*  /ws/*
  v
[ Django + Daphne ]
  |               |
  v               v
[ PostgreSQL ]   [ Redis ]
```

Caddy принимает внешние HTTPS-запросы и автоматически выпускает TLS-сертификат Let's Encrypt. **nginx** отдаёт собранное React-приложение и статические файлы (видео, превью), а обращения к `/api/` и `/ws/` проксирует на серверное приложение. **Django** с ASGI-сервером **Daphne** обслуживает REST-API и WebSocket-соединения, обращаясь к **PostgreSQL** за данными и к **Redis** как к слою каналов комнат.

Серверное приложение разделено на пять Django-приложений по принципу единственной ответственности:

- `accounts` — пользователи, аутентификация, гостевые аккаунты, выдача JWT;
- `videos` — видео, загрузка, потоковая отдача;
- `rooms` — комнаты, участники, синхронизация плеера;
- `chat` — сообщения чата, лайки, вопросы и ответы;
- `common` — общий код: базовые модели, разрешения, постраничная навигация, отдача файлов с поддержкой Range.

2.2 Модель данных

База данных описана классами-моделями Django. Основные сущности:

- **User** — пользователь. Идентификация по имени пользователя, вход по паролю. Хранит необязательный e-mail, отдельное <<имя в чате>>, признак гостевого аккаунта и отметку времени последней активности.
- **Video** — видеоролик: владелец, название, описание, файл, превью, длительность, размер, число просмотров, признак публичности. Первичный ключ — UUID.
- **WatchRoom** — комната просмотра: видео, ведущий и состояние плеера (позиция, флаг воспроизведения, отметка времени). UUID комнаты служит токеном ссылки-приглашения.
- **RoomParticipant** — участник комнаты: зарегистрированный пользователь или гость, роль и признак присутствия в сети.
- **ChatMessage, MessageLike** — сообщения чата и лайки к ним.
- **Question, Answer, QuestionUpvote** — вопросы вкладки <<Вопрос / ответ>> ответы и голоса.

Общие поля (отметки создания и изменения, UUID-ключ) вынесены в абстрактные базовые модели приложения `common` и наследуются остальными моделями — это исключает дублирование кода (принцип DRY).

2.3 REST-API

REST-API смонтирован под префиксом `/api/` и сгруппирован по областям:

- **Авторизация** (`/api/auth/`): регистрация, вход по паролю, гостевой вход, обновление токена, чтение и правка профиля.
- **Видео** (`/api/videos/`): лента, загрузка, карточка ролика, потоковая отдача файла, список своих видео, правка и удаление.
- **Комнаты** (`/api/rooms/`): создание комнаты, чтение состояния по ссылке, список своих комнат, история чата и вопросов.

Полное описание методов с примерами запросов `curl` оформлено в репозитории в файле `API.md`.

2.4 WebSocket-протокол комнаты

Каждая комната обслуживается одним WebSocket-соединением на участника по адресу `/ws/rooms/<id>/`. Так как браузерный WebSocket не передаёт заголовки авторизации, JWT-токен передаётся параметром строки запроса; промежуточный слой проверяет его и определяет пользователя. Гость подключается без токена и может только наблюдать.

Сообщения протокола (формат JSON, поле `type`):

- от ведущего — `player.play`, `player.pause`, `player.seek`;
- от любого участника — `player.sync_request`;
- от авторизованного участника — `chat.message`, `chat.like`, `qa.question`, `qa.answer`, `qa.upvote`;
- от сервера группе — `room.state` (состояние плеера), `room.participants` (состав), а также события чата и вопросов.

2.5 Окружение разработки и развёртывания

Конфигурация приложения управляется переменными окружения. Это позволяет одному и тому же коду работать в двух режимах:

- **локальная разработка** — без внешних сервисов: база данных SQLite и слой каналов в памяти процесса;
- **продакшен** — PostgreSQL и Redis; значения задаются в файле `.env`, который не попадает в репозиторий.

Развёртывание автоматизировано. Образы серверной и клиентской частей описаны в `Dockerfile`; файл `docker-compose.yml` поднимает весь стек одной командой. Скрипт `deploy.sh` разворачивает платформу на сервере с уже настроенным Caddy: он синхронизирует исходники, добавляет сервисы в общий `docker-compose.yml`, дописывает поддомен в конфигурацию Caddy и поднимает контейнеры. Скрипт идиempотентен — повторный запуск не ломает уже развёрнутую систему.

3 ПРАКТИЧЕСКАЯ РЕАЛИЗАЦИЯ

3.1 Регистрация и вход

Вход в систему выполняется по имени пользователя и паролю. При регистрации пользователь задаёт имя пользователя, пароль и, при желании, отображаемое << имя в чате >> (рисунок 2). Пароль проверяется стандартными валидаторами Django — минимальная длина, непохожесть на имя пользователя, отсутствие в словаре частых паролей; сразу после регистрации учётная запись активна.

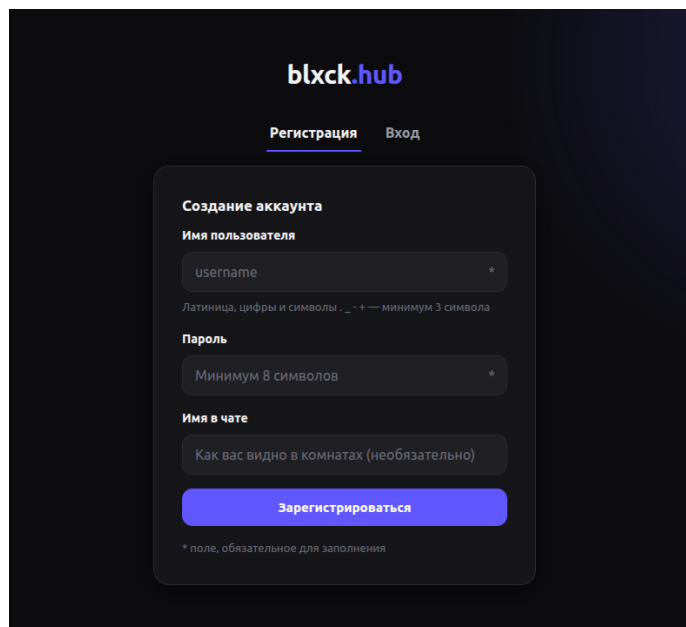


Рисунок 2 — Экран регистрации

Для входа достаточно имени пользователя и пароля (рисунок 3). Кроме того, на экране входа есть кнопка << Войти как гостем >>: она создаёт временный гостевой аккаунт без пароля. Гость пользуется платформой наравне с обычным пользователем — загружает видео, создаёт комнаты, участвует в чате, — но его аккаунт автоматически удаляется после 24 часов простоя вместе со всем содержимым: видео и их файлами, комнатами, сообщениями (рисунок 4). Время последней активности обновляется при каждом запросе и при подключении к комнате, а фоновая команда раз в час удаляет неактивные гостевые аккаунты.

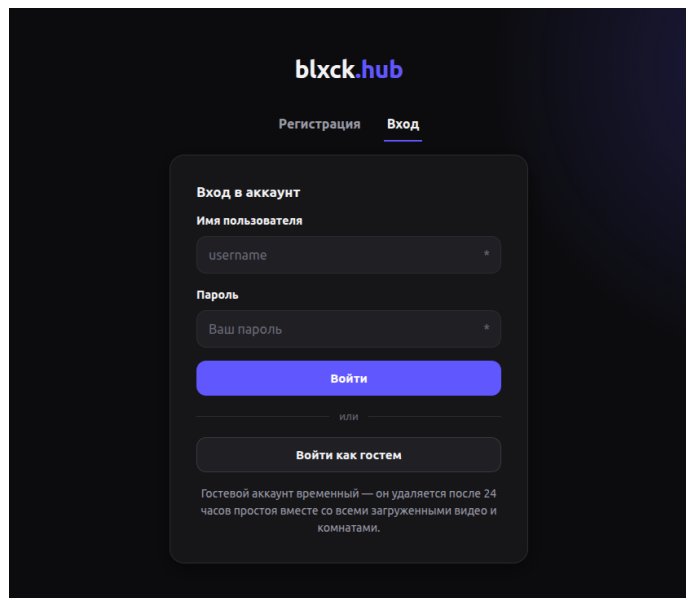


Рисунок 3 — Экран входа: пароль или гостевой доступ

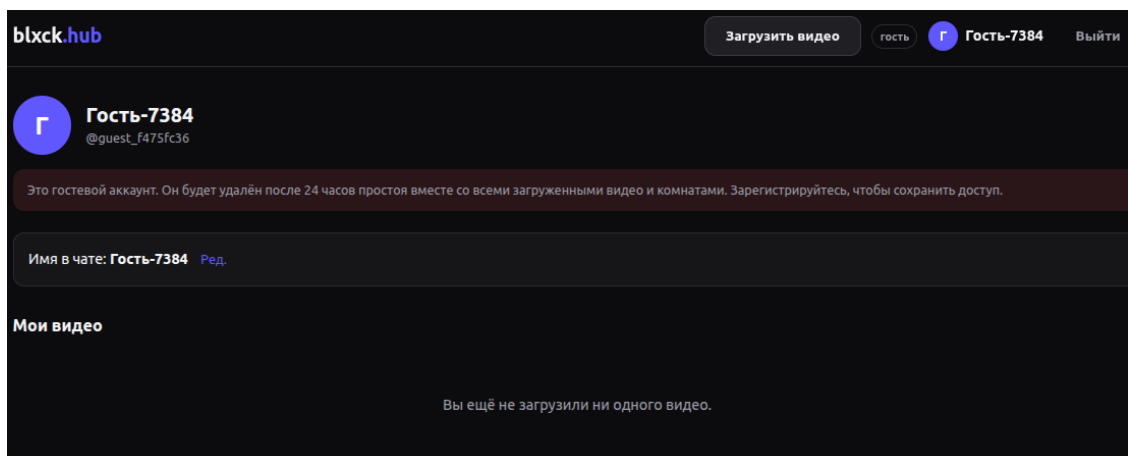


Рисунок 4 — Гостевой аккаунт: профиль с предупреждением об удалении

И при регистрации, и при входе сервер выпускает пару JWT-токенов. Access-токен хранится в памяти клиентского приложения, refresh-токен — в `localStorage`; при ответе сервера `<<401>>` клиент один раз автоматически обновляет access-токен и повторяет запрос.

3.2 Загрузка и потоковая отдача видео

Загрузка видео выполняется на отдельной странице (рисунок 5): пользователь указывает название, описание, выбирает файл и, при желании, превью. Файл передаётся методом `multipart/form-data`; на сервере определяются его размер и длительность (через `ffprobe`).

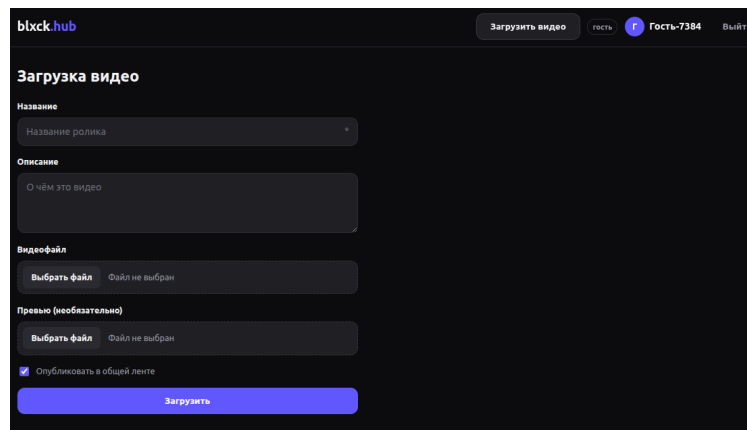


Рисунок 5 — Страница загрузки видео

Потоковая отдача реализована с поддержкой HTTP-заголовка Range: браузерный плеер при перемотке запрашивает не весь файл, а нужный фрагмент. Сервер разбирает заголовок и отвечает кодом <<206 Partial Content>>, передавая запрошенный диапазон байтов. Функция отдачи вынесена в приложение common и переиспользуется.

3.3 Лента, просмотр и профиль

Главная страница показывает ленту опубликованных видео с постраничной подгрузкой (рисунок 6). Страница просмотра содержит плеер, сведения о ролике и кнопку << Смотреть вместе >>, создающую комнату.

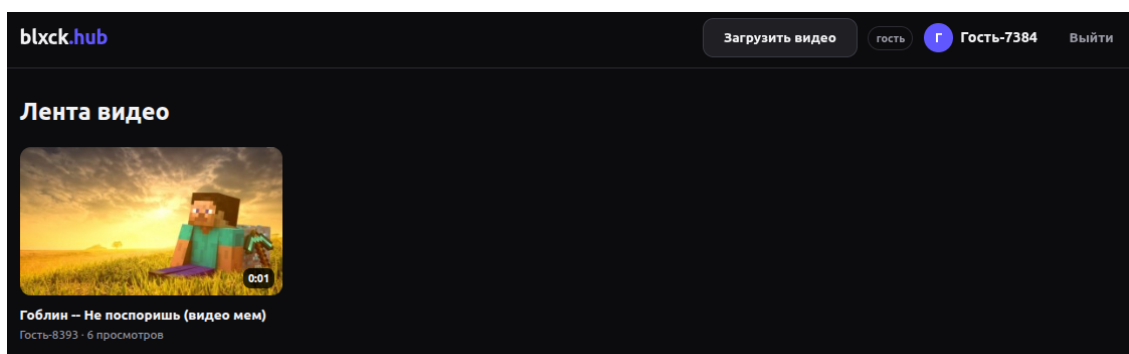


Рисунок 6 — Лента видео

На странице профиля пользователь видит свои видео, может их удалять и редактировать << имя в чате >> (рисунок 8).

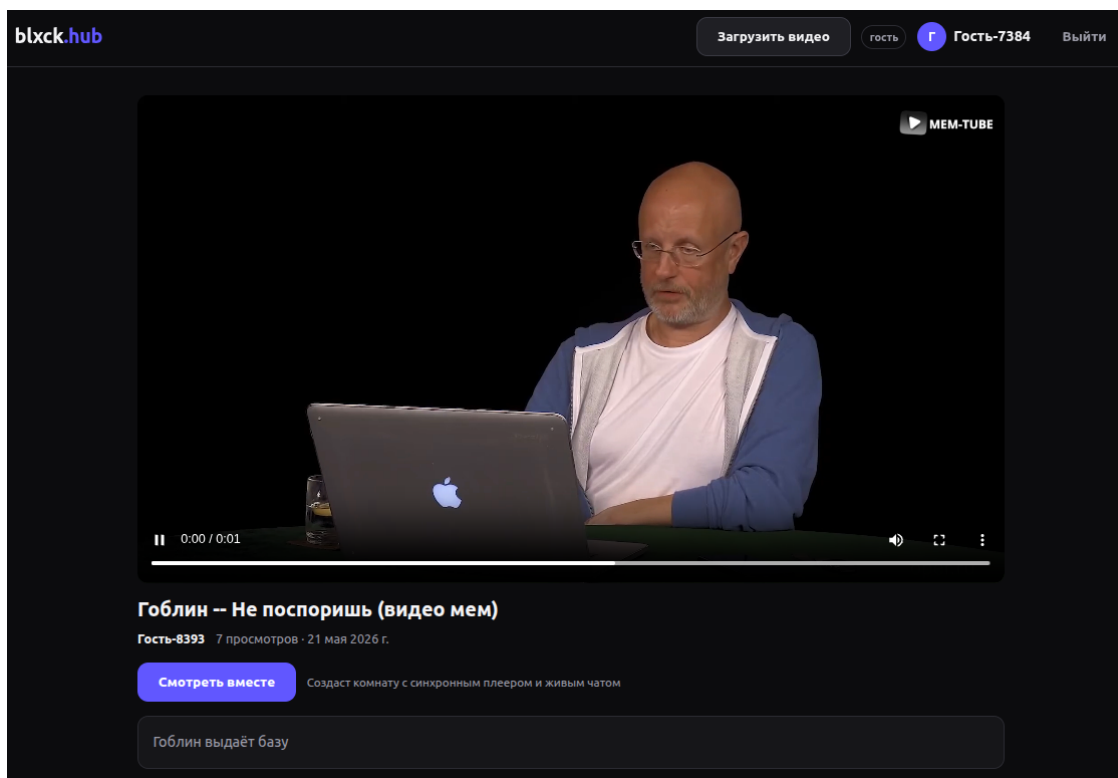


Рисунок 7 — Страница одиночного просмотра видео

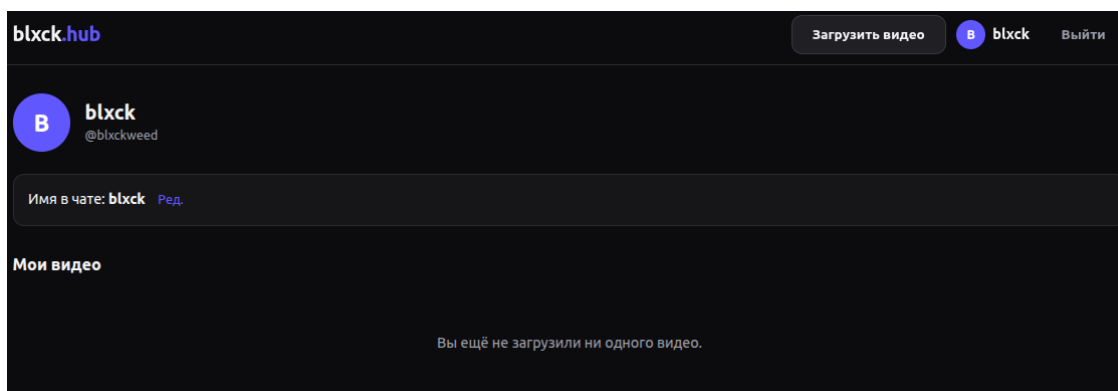


Рисунок 8 — Профиль пользователя и управление своими видео

3.4 Комнаты и синхронизация воспроизведения

Комната совместного просмотра — сигнатурная функция платформы. Её страница повторяет макет Figma: слева плеер, справа панель с чатом и вкладкой вопросов (рисунок 9).

Синхронизацию обеспечивает WebSocket-консьюмер на стороне сервера и хук синхронизации на стороне клиента. Команды ведущего изменяют состояние комнаты в базе и рассылаются всем участникам. Плеер зрителя применяет полученное состояние, а затем каждые три секунды сверяет позицию: расхождение более секунды устраняется перемоткой, меньшее — плав-

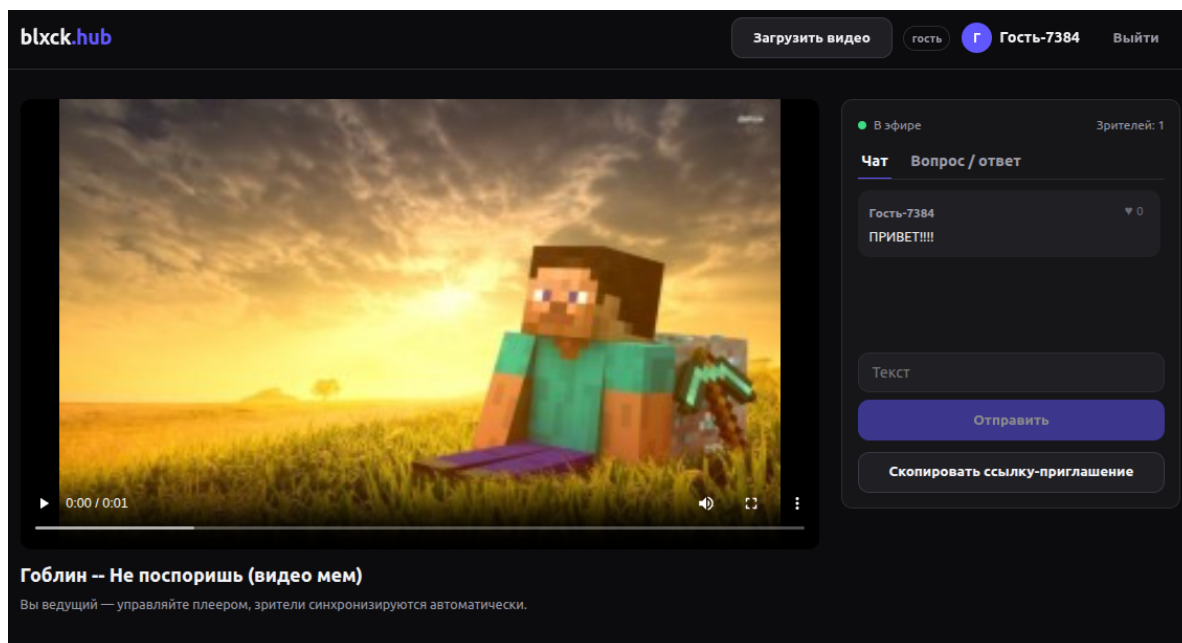


Рисунок 9 — Комната просмотра: плеер и панель чата с лайками

ным изменением скорости. Управление плеером доступно только ведущему, у зрителя оно скрыто.

3.5 Живой чат и вкладка <<Вопрос/ответ>>

Боковая панель комнаты содержит две вкладки. На вкладке <<Чат>> участники обмениваются сообщениями и ставят лайки — сообщение оформлено карточкой из дизайн-системы макета. На вкладке <<Вопрос / ответ>> зрители задают вопросы, отвечают на них и голосуют (рисунок 10). Все события чата идут по тому же WebSocket-соединению, что и синхронизация; анонимный зритель, открывший комнату по ссылке без входа, может читать чат, но не писать в него.

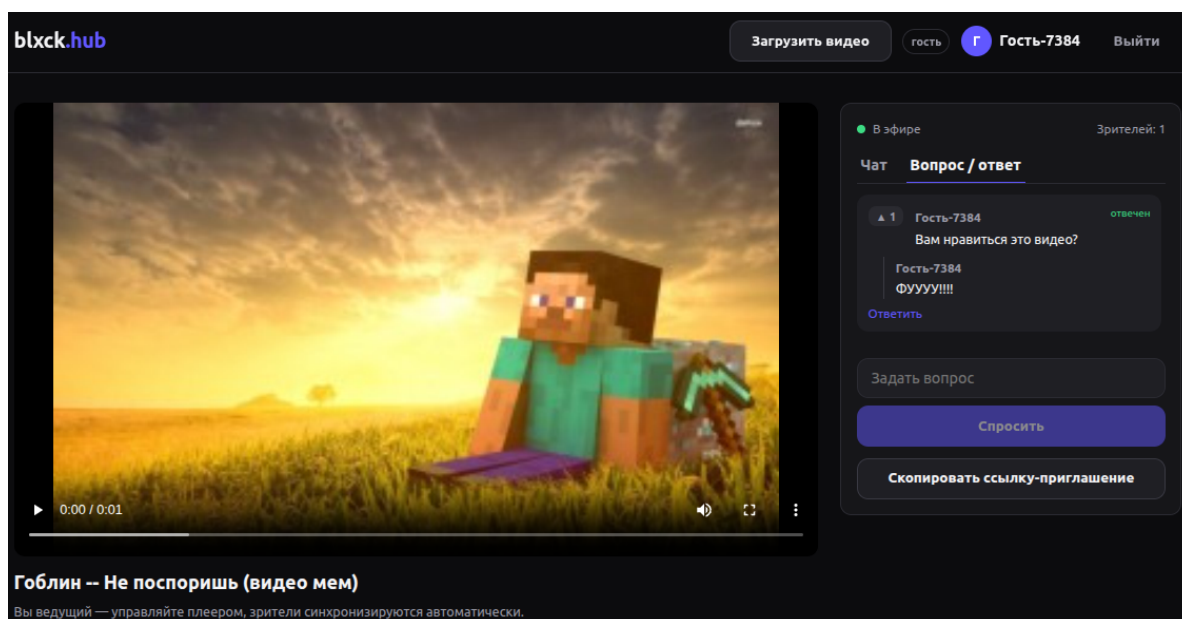


Рисунок 10 — Вкладка <<Вопрос/ответ>> в комнате просмотра

3.6 Клиентское приложение и дизайн-система

Клиентское приложение собрано на React с маршрутизацией React Router. Компоненты дизайн-системы (поле ввода, кнопка, карточка сообщения, вкладки) перенесены из макета Figma и собраны в отдельный каталог. Все цвета, отступы и скругления заданы CSS-переменными — это позволило перекрасить интерфейс под тёмную тему, не трогая разметку компонентов. Вёрстка адаптивная: на узких экранах сетка ленты сворачивается в одну колонку, а панель комнаты переносится под плеер (рисунок 11).

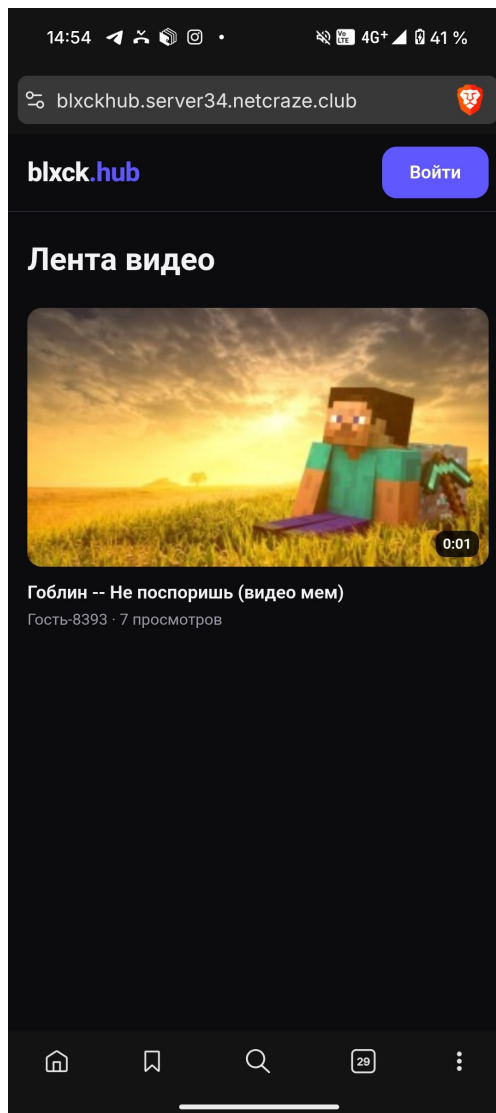


Рисунок 11 — Адаптивная вёрстка на мобильном экране

3.7 Тестирование

Серверная часть покрыта тестами `pytest`: проверяются авторизация (регистрация, вход по паролю, гостевые аккаунты), удаление неактивных гостей вместе с их контентом, загрузка видео и Range-стриминг, чистая функция расчёта позиции воспроизведения, а также WebSocket-консьюмер комнаты — подключение, синхронизация, чат и запрет управления для зрителя (рисунок 12).

Клиентская часть покрыта тестами `Vitest`: проверяются компоненты дизайн-системы, разбор ответов об ошибках и логика коррекции рассинхрона плеера (рисунок 13).

```
pytest - backend

blxck@blxck:~/RGZ/app/backend$ pytest
..... [100%]
41 passed, 1 warning in 14.01s
```

Рисунок 12 — Прогон тестов серверной части (pytest)

```
vitest - frontend

blxck@blxck:~/RGZ/app/frontend$ npm test

> blxck-hub-frontend@1.0.0 test
> vitest run

RUN v4.1.6

Test Files  6 passed (6)
Tests       26 passed (26)
Duration    2.45s
```

Рисунок 13 — Прогон тестов клиентской части (Vitest)

3.8 Развёртывание на сервере

Платформа развёрнута на собственном сервере скриптом `deploy.sh`: он поднимает контейнеры базы данных, Redis, серверной и клиентской частей и подключает поддомен к Caddy (рисунок 14). Доступ к сайту идёт по защищённому протоколу HTTPS с сертификатом Let's Encrypt (рисунок 15).

```
docker compose ps - server

blxck@blxckserver:/opt/stack$ docker compose ps
NAME                IMAGE                                STATUS
blxckhub-backend    stack-blxckhub-backend             Up About an hour
blxckhub-db          postgres:16-alpine                 Up 26 hours
blxckhub-frontend    stack-blxckhub-frontend             Up About an hour
blxckhub-redis       redis:7-alpine                      Up 26 hours
```

Рисунок 14 — Контейнеры blxck.hub, запущенные на сервере

```
HTTPS + TLS certificate

$ curl -sSI https://blxckhub.server34.netcraze.club | head -1
HTTP/2 200

$ openssl s_client -connect blxckhub.server34.netcraze.club:443 \
  -servername blxckhub.server34.netcraze.club </dev/null 2>/dev/null \
  | openssl x509 -noout -issuer -subject -dates
issuer=C = US, O = Let's Encrypt, CN = E8
subject=CN = blxckhub.server34.netcraze.club
notBefore=May 20 04:58:47 2026 GMT
notAfter=Aug 18 04:58:46 2026 GMT
```

Рисунок 15 — Платформа доступна по HTTPS с сертификатом Let's Encrypt

3.9 Трудности и пути их решения

В ходе разработки возникли следующие трудности.

Авторизация по WebSocket. Браузерный WebSocket-API не позволяет задавать HTTP-заголовок `Authorization`, поэтому стандартный способ передачи JWT не работает. Решение — передавать access-токен параметром строки запроса, а на сервере разбирать его в промежуточном слое и подставлять пользователя в сессию соединения.

Настройка коррекции рассинхрона. Жёсткая перемотка плеера зрителя при каждом расхождении приводила к рывкам изображения. Решение — ввести два порога: малый рассинхрон устраняется плавным изменением скорости воспроизведения, и только значительный — перемоткой. Логика вынесена в чистую функцию и покрыта тестами.

Автозапуск видео у зрителя. Браузеры блокируют автоматическое воспроизведение видео со звуком без действия пользователя, из-за чего плеер зрителя не запускался. Решение — показывать поверх плеера кнопку <<Присоединиться к просмотру>>: нажатие является нужным жестом и одновременно запрашивает у сервера актуальное состояние.

Отдача больших видеофайлов. Передача видео целиком на каждый запрос не позволяла перематывать ролик и нагружала сервер. Решение — реализовать поддержку HTTP-заголовка `Range` с ответом <<206 Partial Content>>; статические файлы в продакшене отдаёт nginx.

Поле флага в multipart-форме. При загрузке видео без явного флага публичности сериализатор трактовал отсутствующее булево поле как <<ложь>>, и ролик не попадал в ленту. Решение — разбирать признак публичности отдельно во вьюхе, считая отсутствие значения согласием.

Единая конфигурация для двух окружений. Локальная разработка и продакшен требуют разных баз данных и слоёв каналов. Решение — управлять конфигурацией через переменные окружения: без них приложение использует SQLite и слой каналов в памяти, с ними — PostgreSQL и Redis.

ЗАКЛЮЧЕНИЕ

В ходе расчётно-графической работы спроектирована и реализована тестовая видеоплатформа **blxck.hub** на современном стеке веб-технологий.

Выполнены все поставленные задачи:

- спроектирован в Figma макет интерфейса, его дизайн-система и состав экранов перенесены в код;
- разработана серверная часть на Django и Django REST Framework: модель данных, авторизация по паролю и гостевые аккаунты с выдачей JWT-токенов, загрузка и потоковая отдача видео с поддержкой Range;
- реализована сигнатурная функция — синхронные комнаты совместного просмотра с живым чатом и вкладкой << Вопрос / ответ >> поверх WebSocket и Django Channels;
- разработано клиентское приложение на React с маршрутизацией, контекстом авторизации и компонентами дизайн-системы;
- серверная и клиентская части покрыты автоматическими тестами;
- платформа развёрнута на собственном сервере за обратным прокси Caddy с доступом по HTTPS, документация API оформлена в Git.

В процессе работы получены практические навыки построения клиент-серверного веб-приложения: проектирование REST-API, организация обмена в реальном времени по WebSocket, реализация авторизации на JWT-токенах, разработка одностраничного приложения на React и контейнерное развёртывание.

Кодовая база организована по принципам модульности, DRY и единственной ответственности, снабжена тестами и документацией, поэтому пригодна для дальнейшего расширения. Возможные направления развития: передача роли ведущего другому участнику комнаты, транскодирование загружаемых видео в несколько качеств с адаптивным стримингом, поиск и рекомендации.